

Customer Problem

Using AWS SERVERLESS SERVICES

Customer: Event Organizer

An event management organization is struggling to handle growing registration volumes using a manual system built on Microsoft Forms and Excel spreadsheets. The operations team lacks automated workflows for attendee confirmation emails, real-time system monitoring, and a structured deployment process, creating severe administrative bottlenecks. They urgently need to migrate to a highly scalable, serverless AWS REST API that automates event ticketing, provides strict cost tracking within the free tier, and streamlines developer updates through a CI/CD pipeline.

EVENT REGISTRATION & TICKETING SYSTEM

A SERVERLESS API

Objectives

Design, architect, and build a serverless Event Registration & Ticketing System using AWS Cloud Services. The system replaces Microsoft Forms + Excel with a scalable REST API.

You are free to explore additional tools to enhance the project while producing the expected output.

Project Architecture

Architecture Overview

GitHub Repository

CI/CD Pipeline (GitHub Actions)

API Gateway (REST Endpoints)

λ AWS Lambda (Business Logic)

DynamoDB

Events & Registrations

CloudWatch

Logs & Alarms

SNS (optional)

Confirmation Emails

AWS Budgets

Cost Tracking (Free Tier)

Phased Milestones

Phase 1: Infrastructure Foundation

Activities

- Research Cloud Infrastructure services for static hosting
- Investigate serverless compute options (AWS Lambda)
- Learn about API Gateway services and their role
- Understand Identity and Access Management (IAM) in AWS
- Design a resource template that creates all necessary infrastructure

Phased Milestones

Phase 2: API Development

Challenge: Build the 4 core REST API endpoints

Activities

POST	/register	<i>Register for events</i>
GET	/events	<i>List all events</i>
GET	/registrations/{email}	<i>View registrations</i>
DELETE	/registration/{id}	<i>Cancel registration</i>

- Research DynamoDB table design for events & registrations
- Implement robust error handling and logging
- Learn about CORS for web integration
- Understand function event handling and response formatting

Phased Milestones

Phase 3: Automation & CI/CD

Challenge: Implement CI/CD pipeline using GitHub Actions

Activities

- Set up GitHub repository with proper branching strategy
- Create GitHub Actions workflows to automatically test Lambda functions on push
- Implement automated validation and monitoring of Lambda functions
- Monitor CI/CD pipeline logs in GitHub Actions for build success/failure
- Research deployment automation tools and techniques

Phased Milestones

Phase 4: Monitoring & Security

Challenge: Set up robust monitoring and alarms

Activities

- Track API request count, failed registrations, and Lambda duration
- Create CloudWatch alarm if error rate exceeds 5%
- Implement CloudWatch Logs for all Lambda functions
- Research API security patterns for public endpoints
- Understand the principle of least privilege for IAM permissions
- Implement proper input validation and sanitization
- (Optional) Set up SNS for confirmation emails
- Use AWS Budgets to track cost and stay within Free Tier

Phased Milestones

Phase 5: Deployment and Optimization

Activities

- Research deployment automation tools and techniques
- Investigate monitoring and logging solutions
- Learn about cost optimization strategies for cloud resources
- Understand how to set up proper resource lifecycle policies
- Implement automated deployment scripts

Deliverables:

- GitHub repo with API code
- CI/CD pipeline (GitHub Actions)
- Lambda functions
- DynamoDB table definitions
- CloudWatch alarms config
- README file
- Product presentation (problem, challenges, demo)

Final Outcome

Event Registration & Ticketing System

Event Name

Email Address

AWS Workshop Accra 2026

participant@email.com

Register →

Available Events

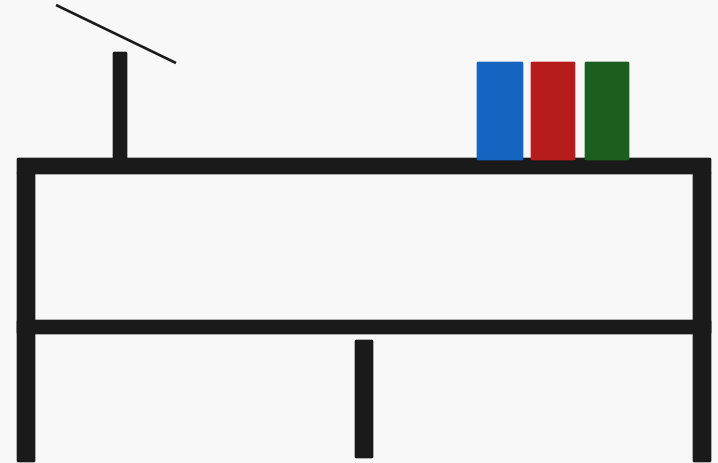
AWS Workshop Accra 2026	15 May 2026	Available
Cloud Solutions Summit	28 Jun 2026	Limited

Self-study Material Links

Links to videos, resources and documentation

For quick references and guides, here are introductory and practical resources to assist you along the way.

- [GitHub Actions Documentation](#)
- [AWS Lambda Documentation](#)
- [Amazon API Gateway](#)
- [Amazon DynamoDB](#)
- [Amazon CloudWatch](#)
- [Amazon SNS](#)
- [AWS Budgets](#)
- [Boto3 \(AWS SDK for Python\)](#)
- [AWS CloudFormation](#)
- [AWS SAM](#)



Links to Resources and Documentation

Links to videos, resources and documentation

Documentation for tools used in this project

GitHub Actions <https://docs.github.com/en/actions>

AWS Lambda <https://docs.aws.amazon.com/lambda>

API Gateway <https://docs.aws.amazon.com/apigateway>

DynamoDB <https://docs.aws.amazon.com/dynamodb>

CloudWatch <https://docs.aws.amazon.com/cloudwatch>

IAM <https://docs.aws.amazon.com/iam>

SNS <https://docs.aws.amazon.com/sns>

Remember: The goal is not just to build the application, but to understand the why behind each architectural decision.

Focus on learning the principles that will apply to future projects!